

Vektoriaus klasė

Generated by Doxygen 1.13.2

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Vector< T > Class Template Reference	5
4 File Documentation	7
4.1 vektorius.h	7
Index	13

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Vector< T >	5
-----------------------------------	---

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

vektorius.h		7
-----------------------------	-------	---

Chapter 3

Class Documentation

3.1 `Vector< T >` Class Template Reference

Public Member Functions

- `Vector` (`std::initializer_list< T >` list)
- `Vector` & `operator=` (`std::initializer_list< T >` list)
- `Vector` (`const Vector` &v)
- `Vector` & `operator=` (`const Vector` &v)
- `Vector` (`Vector` &&v) noexcept
- `Vector` & `operator=` (`Vector` &&v) noexcept
- `T * begin` ()
- `const T * begin` () const
- `T * end` ()
- `const T * end` () const
- `T * rbegin` ()
- `T * rend` ()
- `const T * cbegin` () const
- `const T * cend` () const
- `T & front` ()
- `T & back` ()
- `T * data` ()
- `T * data` () const
- `size_t size` ()
- `size_t capacity` ()
- `bool empty` () const
- `void push_back` (`const T` &elementas)
- `void pop_back` ()
- `void swap` (`Vector` &v)
- `void swap` (`T` &a, `T` &b)
- `void clear` ()
- `void assign` (`size_t` kiekis, `const T` &elementas)
- `void assign` (`T *begin`, `T *end`)
- `void assign` (`std::initializer_list< T >` list)
- `void resize` (`size_t` naujaTalpa)
- `void resize` (`size_t` naujaTalpa, `const T` &elementas)
- `void reserve` (`size_t` naujaTalpa)
- `void shrink_to_fit` ()

- void **erase** (T *index)
- void **erase** (T *begin, T *end)
- void **insert** (T *index, const T &elementas)
- T & **at** (size_t index)
- const T & **at** (size_t pos) const
- void **append_range** (std::initializer_list< T > list)
- T & **operator[]** (size_t index)
- bool **operator==** (const [Vector](#) &v) const
- bool **operator!=** (const [Vector](#) &v) const

The documentation for this class was generated from the following file:

- vektorius.h

Chapter 4

File Documentation

4.1 vektorius.h

```
00001 #include "../mano_lib.h"
00002 template <typename T>
00003 class Vector
00004 {
00005 private:
00006     T *duomenys = nullptr;
00007     size_t dydis = 0;
00008     size_t talpa = 0;
00009
00010     void resize()
00011     {
00012         if (talpa==0) talpa=1;
00013         else talpa *= 2;
00014         {
00015             T *naujiDuomenys = new T[talpa];
00016             for (size_t i = 0; i < dydis; ++i)
00017             {
00018                 naujiDuomenys[i] = duomenys[i];
00019             }
00020             delete[] duomenys;
00021             duomenys = naujiDuomenys;
00022         }
00023     }
00024
00025 public:
00026     // konstruktoriai destruktoriai
00027     Vector() = default;
00028
00029     Vector(std::initializer_list<T> list) : talpa(list.size()), dydis(list.size())
00030     {
00031         duomenys = new T[talpa];
00032         std::copy(list.begin(), list.end(), duomenys);
00033     }
00034
00035     Vector &operator=(std::initializer_list<T> list)
00036     {
00037         talpa = list.size();
00038         duomenys = new T[talpa];
00039         std::copy(list.begin(), list.end(), duomenys);
00040         dydis = talpa;
00041         return *this;
00042     }
00043
00044     ~Vector()
00045     {
00046         delete[] duomenys;
00047         dydis = 0;
00048         talpa = 0;
00049     }
00050
00051     // copy constructor
00052     Vector(const Vector &v) : dydis(v.dydis), talpa(v.talpa)
00053     {
00054         duomenys = new T[talpa];
00055         for (size_t i = 0; i < dydis; i++)
00056         {
00057             duomenys[i] = v.duomenys[i];
```

```

00058     }
00059 }
00060
00061 // copy assignment operator
00062 Vector &operator=(const Vector &v)
00063 {
00064     if (this != &v)
00065     {
00066         delete[] duomenys;
00067         dydis = v.dydis;
00068         talpa = v.talpa;
00069         duomenys = new T[talpa];
00070         for (size_t i = 0; i < dydis; i++)
00071         {
00072             duomenys[i] = v.duomenys[i];
00073         }
00074     }
00075     return *this;
00076 }
00077
00078 // move constructor
00079 Vector(Vector &&v) noexcept : duomenys(v.duomenys), talpa(v.talpa), dydis(v.dydis)
00080 {
00081     v.duomenys = nullptr;
00082     v.talpa = 0;
00083     v.dydis = 0;
00084 }
00085
00086 // move assignment operator
00087 Vector &operator=(Vector&& v) noexcept
00088 {
00089     if (this != &v)
00090     {
00091         delete[] duomenys;
00092         dydis = v.dydis;
00093         talpa = v.talpa;
00094         {
00095             duomenys = new T[talpa];
00096             for (size_t i = 0; i < dydis; i++)
00097             {
00098                 duomenys[i] = v.duomenys[i];
00099             }
00100         }
00101     }
00102     v.duomenys = nullptr;
00103     v.talpa = 0;
00104     v.dydis = 0;
00105     return *this;
00106 }
00107
00108 // grazinamos reikšmes
00109
00110 T *begin() { return duomenys; }
00111 const T *begin() const { return duomenys; }
00112
00113 T *end() { return duomenys + dydis; }
00114 const T *end() const { return duomenys + dydis; }
00115
00116 T *rbegin()
00117 {
00118     if (dydis == 0)
00119         return duomenys;
00120     else
00121         return duomenys + dydis - 1;
00122 }
00123
00124 T *rend()
00125 {
00126     if (dydis == 0)
00127         return duomenys;
00128     else
00129         return duomenys - 1;
00130 }
00131
00132 const T *cbegin() const { return duomenys; }
00133 const T *cend() const { return duomenys + dydis; }
00134
00135 T &front()
00136 {
00137     return this->at(0);
00138 }
00139
00140 T &back()
00141 {
00142     return this->at(dydis - 1);
00143 }
00144

```

```
00145     T *data()
00146     {
00147         return duomenys;
00148     }
00149
00150     T *data() const
00151     {
00152         return duomenys;
00153     }
00154
00155     size_t size()
00156     {
00157         return dydis;
00158     }
00159
00160     size_t capacity()
00161     {
00162         return talpa;
00163     }
00164
00165     bool empty() const
00166     {
00167         if (dydis == 0)
00168             return true;
00169         else
00170             return false;
00171     }
00172
00173     void push_back(const T &elementas)
00174     {
00175         if (dydis == talpa) resize();
00176         duomenys[dydis] = elementas;
00177         ++dydis;
00178     }
00179
00180     void pop_back()
00181     {
00182         if (dydis > 0)
00183         {
00184             --dydis;
00185         }
00186     }
00187
00188     void swap(Vector &v)
00189     {
00190         std::swap(duomenys, v.duomenys);
00191         std::swap(dydis, v.dydis);
00192         std::swap(talpa, v.talpa);
00193     }
00194
00195     void swap( T& a, T& b )
00196     {
00197         std::swap(a,b);
00198     }
00199
00200     void clear()
00201     {
00202         dydis = 0;
00203         talpa = 0;
00204         delete[] duomenys;
00205         duomenys = nullptr;
00206     }
00207
00208     void assign(size_t kiekis, const T &elementas)
00209     {
00210         if (kiekis > talpa)
00211             resize(kiekis);
00212         std::fill_n(duomenys, kiekis, elementas);
00213         dydis = kiekis;
00214     }
00215
00216     void assign(T *begin, T *end)
00217     {
00218         size_t kiekis = std::distance(begin, end);
00219         if (kiekis > talpa)
00220             resize(kiekis);
00221         std::copy(begin, end, duomenys);
00222         dydis = kiekis;
00223     }
00224
00225     void assign(std::initializer_list<T> list)
00226     {
00227         size_t kiekis = list.size();
00228         if (kiekis > talpa)
00229             resize(kiekis);
00230         std::copy(list.begin(), list.end(), duomenys);
00231         dydis = kiekis;
```

```

00232     }
00233
00234     void resize(size_t naujaTalpa)
00235     {
00236         talpa = naujaTalpa;
00237         T *naujiDuomenys = new T[talpa];
00238         for (size_t i = 0; i < dydis; ++i)
00239         {
00240             naujiDuomenys[i] = duomenys[i];
00241         }
00242         delete[] duomenys;
00243         duomenys = naujiDuomenys;
00244     }
00245
00246     void resize(size_t naujaTalpa, const T &elementas)
00247     {
00248         talpa = naujaTalpa;
00249         T *naujiDuomenys = new T[talpa];
00250         for (size_t i = 0; i < dydis; ++i)
00251         {
00252             naujiDuomenys[i] = duomenys[i];
00253         }
00254         for (size_t i = dydis; i < talpa; ++i)
00255         {
00256             naujiDuomenys[i] = elementas;
00257         }
00258         dydis = talpa;
00259         delete[] duomenys;
00260         duomenys = naujiDuomenys;
00261     }
00262
00263     void reserve(size_t naujaTalpa)
00264     {
00265         resize(naujaTalpa);
00266     }
00267
00268     void shrink_to_fit()
00269     {
00270         if (dydis < talpa)
00271             resize(dydis);
00272     }
00273
00274     void erase(T *index)
00275     {
00276         size_t vieta = index - duomenys;
00277         if (index > duomenys + dydis || index < duomenys || vieta < 0)
00278         {
00279             throw std::out_of_range("Out of range: " + std::to_string(vieta) + " >= this->size: " +
std::to_string(this->dydis));
00280         }
00281         for (size_t i = vieta; i + 1 < dydis; ++i)
00282         {
00283             duomenys[i] = duomenys[i + 1];
00284         }
00285         --dydis;
00286     }
00287
00288     void erase(T *begin, T *end)
00289     {
00290         size_t kiekis = std::distance(begin, end);
00291         size_t vieta = begin - duomenys;
00292         for (size_t i = vieta; i + kiekis < dydis; ++i)
00293         {
00294             duomenys[i] = duomenys[i + kiekis];
00295         }
00296         dydis -= kiekis;
00297     }
00298
00299     void insert(T *index, const T &elementas)
00300     {
00301         size_t vieta = index - duomenys;
00302         if (dydis == talpa)
00303             resize();
00304         for (size_t i = dydis; i > vieta; i--)
00305         {
00306             duomenys[i] = duomenys[i - 1];
00307         }
00308         duomenys[vieta] = elementas;
00309         dydis++;
00310     }
00311
00312
00313     T &at(size_t index)
00314     {
00315         if (index >= dydis || index < 0)
00316         {
00317             throw std::out_of_range("Out of range: " + std::to_string(index) + " >= this->size: " +

```

```
std::to_string(this->dydis));
00318     }
00319     return duomenys[index];
00320 }
00321
00322     const T& at(size_t pos) const {
00323         if (pos >= dydis) {
00324             throw std::out_of_range("Out of range: " + std::to_string(pos) + " >= this->size: " +
std::to_string(this->dydis));
00325         }
00326         return duomenys[pos];
00327     }
00328
00329     void append_range(std::initializer_list<T> list)
00330     {
00331         if (dydis + list.size() > talpa)
00332         {
00333             reserve(dydis + list.size());
00334         }
00335         for (auto &elementas : list)
00336         {
00337             duomenys[dydis++] = elementas;
00338         }
00339     }
00340
00341     // operatoriu overloadai
00342     T &operator[](size_t index)
00343     {
00344         if (index >= dydis)
00345         {
00346             throw std::out_of_range("Index out of bounds");
00347         }
00348         return duomenys[index];
00349     }
00350
00351     bool operator==(const Vector&v) const{
00352         if(dydis != v.dydis) return false;
00353         for(int i=0; i<dydis; i++)
00354         {
00355             if (duomenys[i] != v.duomenys[i]) return false;
00356         }
00357         return true;
00358     }
00359
00360     bool operator!=(const Vector&v) const{
00361         if(dydis != v.dydis) return true;
00362         for(int i=0; i<dydis; i++)
00363         {
00364             if (duomenys[i] != v.duomenys[i]) return true;
00365         }
00366         return false;
00367     }
00368 };
```


Index

Vector< T >, [5](#)